

SMC PLIC — Low-Level Design

SMC PLIC — Low-Level SystemC Implementation

Document: `02_PLIC_LowLevel_Design.md` **Module:** `smc::plic` (`peripherals/plic/`) **SystemC:** 3.0.2 Accellera (compatible with $\geq 2.3.4$) **Status:** Code-validated; matches `peripherals/plic/include/plic.h` and `src/plic.cpp` **Companion docs:** - `01_PLIC_Specification.md` — externally-observable behaviour - `03_PLIC_Test_Plan.md` — verification strategy & test list

Contents

1. Purpose & scope
 2. Source layout
 3. Module structure
 - 3.1 Ports
 - 3.2 Public methods (API)
 - 3.3 Internal methods and SC processes
 - 3.4 File-local helpers
 - 3.5 Function call flow
 4. TLM-2.0 interface implementation
 5. Register decode
 6. Internal data structures
 7. Process / event topology — single-driver discipline
 8. Behavioural algorithms
 9. Reset implementation
 10. Loosely-timed timing model
 11. Error handling implementation
 12. Debug & verification hooks
 13. Integration with `smc_top`
 14. Modeling decisions & trade-offs
 15. Performance & footprint
 16. Known limitations & future work
 17. Code walkthrough — function reference
 18. Build, packaging and dependency notes
-

1. Purpose & scope

`01_PLIC_Specification.md` defines **what** the PLIC looks like from the outside — sockets, signals, register layout, protocol semantics. This document defines **how** the SystemC model is implemented internally:

- the data structures it owns,

- the SystemC process topology that drives them,
- the algorithms used for source latching, arbitration, and claim/complete,
- the design decisions taken to satisfy SystemC 3.0's strict single-driver rule, the RISC-V PLIC v1.0 spec, and the SMC's loosely-timed budget,
- the verification hooks exposed to the test bench, and
- the exact integration recipe required to drop the model into `smc_top` and run firmware against it.

It is the implementation contract that any future maintainer (or an alternate implementation that wants to remain bit-compatible with the SMC PLIC) should be able to read in isolation.

2. Source layout

```
peripherals/plic/
├── CMakeLists.txt
├── README.md
├── run_tests.sh           Build + run convenience script
├── doc/
│   ├── 01_PLIC_Specification.md
│   ├── 02_PLIC_LowLevel_Design.md (this file)
│   └── 03_PLIC_Test_Plan.md
├── include/
│   ├── plic.h             SC_MODULE(plic) declaration
│   └── smc_tlm_extensions.h Shared GP extension (smc_axi_extension)
├── src/
│   └── plic.cpp            Implementation
├── test/
│   ├── CMakeLists.txt
│   └── plic_tb.cpp        Self-checking test bench
└── build/                 (out-of-source CMake build tree)
```

Total payload: ~970 lines of C++17 incl. tests, ~520 lines excl. tests.

3. Module structure

```
#include <cci_configuration> // OSCI CCI - cci_param, cci_broker_handle

namespace smc {

struct plic_cfg {
    unsigned num_sources = 336; // 1..1023 (CCI default)
    unsigned num_contexts = 8; // 4 cores × {M, S} (CCI default)

    static constexpr uint64_t PRIORITY_BASE = 0x000000;
    static constexpr uint64_t PENDING_BASE = 0x001000;
    static constexpr uint64_t ENABLE_BASE = 0x002000;
    static constexpr uint64_t ENABLE_STRIDE = 0x000080;
    static constexpr uint64_t CONTEXT_BASE = 0x200000;
    static constexpr uint64_t CONTEXT_STRIDE = 0x001000;
    static constexpr uint64_t CONTEXT_THR_OFF = 0x000000;
    static constexpr uint64_t CONTEXT_CC_OFF = 0x000004;
    static constexpr uint64_t WINDOW_SIZE = 0x400000; // 4 MB
};

class plic : public sc_core::sc_module {
protected:
    // CCI params - declared BEFORE sc_vector ports so they are
    // initialised first; src_in/ctx_out sizes are read from these.
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> num_sources_p_;
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> num_contexts_p_;
    cci::cci_param<double> access_delay_ns_p_;

public:
    SC_HAS_PROCESS(plic);

    tlm_utils::simple_target_socket<plic> reg_socket;
    sc_core::sc_vector<sc_core::sc_in<bool>> src_in; // sized from num_sources_p_
    sc_core::sc_vector<sc_core::sc_out<bool>> ctx_out; // sized from num_contexts_p_
    sc_core::sc_in<bool> rst_n_i;

    explicit plic(sc_core::sc_module_name name, plic_cfg cfg = plic_cfg{});

    // Test-bench back-door (no socket, no side effects)
    uint32_t dbg_priority (unsigned src) const;
    bool dbg_pending (unsigned src) const;
    bool dbg_enable (unsigned ctx, unsigned s) const;
    uint32_t dbg_threshold (unsigned ctx) const;
    uint32_t dbg_claim_top (unsigned ctx) const;
    void dump_state (std::ostream& = std::cout) const;

private:
    void b_transport (tlm::tlm_generic_payload&, sc_core::sc_time&);
    unsigned int transport_dbg(tlm::tlm_generic_payload&);

    void reset_proc(); // SC_METHOD on rst_n_i
    void src_method(); // SC_METHOD on every src_in[i]
    void output_method(); // SC_METHOD on recompute_event_
    void schedule_recompute();

    bool reg_read (uint64_t off, uint32_t& data);
    bool reg_write(uint64_t off, uint32_t data);

    uint32_t claim (unsigned ctx);
    void complete(unsigned ctx, uint32_t src);
    uint32_t best_pending(unsigned ctx) const;
};
```

```

plic_cfg cfg_;
std::vector<uint32_t> priority_; // synced from CCI params at ctor
std::vector<bool> pending_; // [num_sources+1]
std::vector<bool> last_level_; // edge-detect helper
std::vector<bool> claim_in_flight_; // [num_sources+1]
std::vector<std::vector<uint32_t>> enable_; // [ctx][word]
std::vector<uint32_t> threshold_; // [num_contexts]
std::vector<bool> ctx_out_cache_; // last-driven ctx_out value
sc_core::sc_event recompute_event_;
// access_delay_ns_p_ (CCI param above) replaces the old sc_time access_delay_ field.
};

} // namespace smc

```

Why each choice on this list:

- **SC_HAS_PROCESS + class (not the SC_MODULE macro)** — the macro pulls in a default constructor; we need to forward `plic_cfg` so the field vectors can be sized from constructor arguments.
- **`tlm_utils::simple_target_socket`** — matches every other SMC IP per §6.1 of the architecture document. Default data width 64-bit is fine because we reject any `data_length != 4`.
- **`sc_vector<sc_in<bool>> for sources`** — lets the size be passed at construction time and exposes individual `[i]` ports for binding from every SMC peripheral's `irq_o`.
- **`sc_in<bool> rst_n_i`** — single boolean reset, sampled level-low per the convention of every other SMC IP.
- **One STL vector per state class** — over a fixed-size `std::array`, because the size depends on `cfg_`. Number of words used by `enable_` is `ceil((num_sources+1)/32)`; for the spec defaults that is 11 words per context.
- **`ctx_out_cache_`** — `sc_signal` change events only fire when the written value differs from the current value; nevertheless we maintain an explicit cache so that `output_method` is idempotent and cheap (no spurious socket activity, predictable wake patterns).
- **CCI params declared protected before public ports** — CCI params are member-initialised in declaration order; `src_in` and `ctx_out` sizes are read from `num_sources_p_` and `num_contexts_p_` at construction, so the params must be fully initialised first. `protected` visibility lets a derived test-bench subclass inspect or override them if needed.
- **`num_sources_p_ / num_contexts_p_ as CCI_IMMUTABLE_PARAM`** — the `sc_vector` sizes cannot change after elaboration; making the params immutable enforces this invariant and raises a CCI error if any tool or TB tries to alter them post-elaboration.
- **`access_delay_ns_p_ as a plain mutable cci_param<double>`** — the delay is not structural; it can be legitimately adjusted between transactions (e.g. to model different fabric speeds for regression sets) without breaking any invariant.

3.1 Ports

Port	Type	Dir	Purpose
<code>reg_socket</code>	<code>tlm_utils::simple_target_socket<plic></code>	target	AXI4-Lite-style TLM-2.0 register access.
<code>src_in[i]</code>	<code>sc_core::sc_vector<sc_core::sc_in<bool>></code>	in	Interrupt source <code>i</code> ($\rightarrow$ source ID <code>i+1</code>); active-high, level-sensitive; sized from <code>num_sources</code> .
<code>ctx_out[c]</code>	<code>sc_core::sc_vector<sc_core::sc_out<bool>></code>	out	Interrupt output for context <code>c</code> ; active-high; sized from <code>num_contexts</code> .
<code>rst_n_i</code>	<code>sc_core::sc_in<bool></code>	in	Active-low synchronous reset.

3.2 Public methods (API)

Method	Signature	Purpose
constructor	<code>plic(sc_module_name, plic_cfg = {})</code>	Build, resolve CCI, size vectors/ports, register callbacks/processes (see §3 ctor + §7).
<code>dbg_priority</code>	<code>uint32_t dbg_priority(unsigned src) const</code>	Back-door read of a source's 3-bit priority (0 if OOR).
<code>dbg_pending</code>	<code>bool dbg_pending(unsigned src) const</code>	Back-door query of a source's pending bit.
<code>dbg_enable</code>	<code>bool dbg_enable(unsigned ctx, unsigned src) const</code>	Back-door query of the <code>(ctx, src)</code> enable bit.
<code>dbg_threshold</code>	<code>uint32_t dbg_threshold(unsigned ctx) const</code>	Back-door read of a context's 3-bit threshold.
<code>dbg_claim_top</code>	<code>uint32_t dbg_claim_top(unsigned ctx) const</code>	Top eligible pending source for <code>ctx</code> without claiming (§8, §12).
<code>dump_state</code>	<code>void dump_state(std::ostream& = std::cout) const</code>	Human-readable snapshot of all PLIC state.

3.3 Internal methods and SC processes

Member	Signature	Kind	Purpose
<code>b_transport</code>	<code>void b_transport(tlm::tlm_generic_payload&, sc_core::sc_time&)</code>	TLM <code>b_transport</code> callback	Blocking register access; a CLAIM read has the <code>claim()</code> side effect (§4).
<code>transport_dbg</code>	<code>unsigned int transport_dbg(tlm::tlm_generic_payload&)</code>	TLM <code>transport_dbg</code> callback	Side-effect-free back-door access (CLAIM read does not claim) (§4, §12).
<code>reset_proc</code>	<code>void reset_proc()</code>	SC_METHOD (sensitive to <code>rst_n_i</code>)	Zero all registers on assertion; recompute to deassert <code>ctx_out</code> (§9).
<code>src_method</code>	<code>void src_method()</code>	SC_METHOD (sensitive to <code>src_in</code>)	Latch rising-edge source events into <code>pending_</code> (§8).
<code>output_method</code>	<code>void output_method()</code>	SC_METHOD (driven by <code>recompute_event_</code>)	Sole driver of all <code>ctx_out[*]</code> (§7).
<code>schedule_recompute</code>	<code>void schedule_recompute()</code>	private helper	Post <code>recompute_event_</code> at <code>SC_ZERO_TIME</code> (§7).
<code>reg_read</code>	<code>bool reg_read(uint64_t off, uint32_t& data)</code>	private helper	Decode a 32-bit register read; CLAIM read calls <code>claim()</code> (§5).
<code>reg_write</code>	<code>bool reg_write(uint64_t off, uint32_t data)</code>	private helper	Decode a 32-bit register write; CLAIM/COMPLETE write calls <code>complete()</code> (§5).
<code>claim</code>	<code>uint32_t claim(unsigned ctx)</code>	private helper	Atomically claim the top eligible source for <code>ctx</code> (§8).
<code>complete</code>	<code>void complete(unsigned ctx, uint32_t src)</code>	private helper	Clear in-flight latch; re-arm <code>pending_</code> if line still high (§8).
<code>best_pending</code>	<code>uint32_t best_pending(unsigned ctx) const</code>	private helper	Highest-priority eligible source for <code>ctx</code> ; low-ID tie-break (§8).

Processes/threads: the model registers exactly **three** SC processes — `reset_proc`, `src_method`, and `output_method`, all **SC_METHOD**. There are **no** **SC_THREAD** / **SC_CTHREAD** processes. `output_method` is the **only** writer of the `ctx_out` signals; every other path mutates internal state and calls

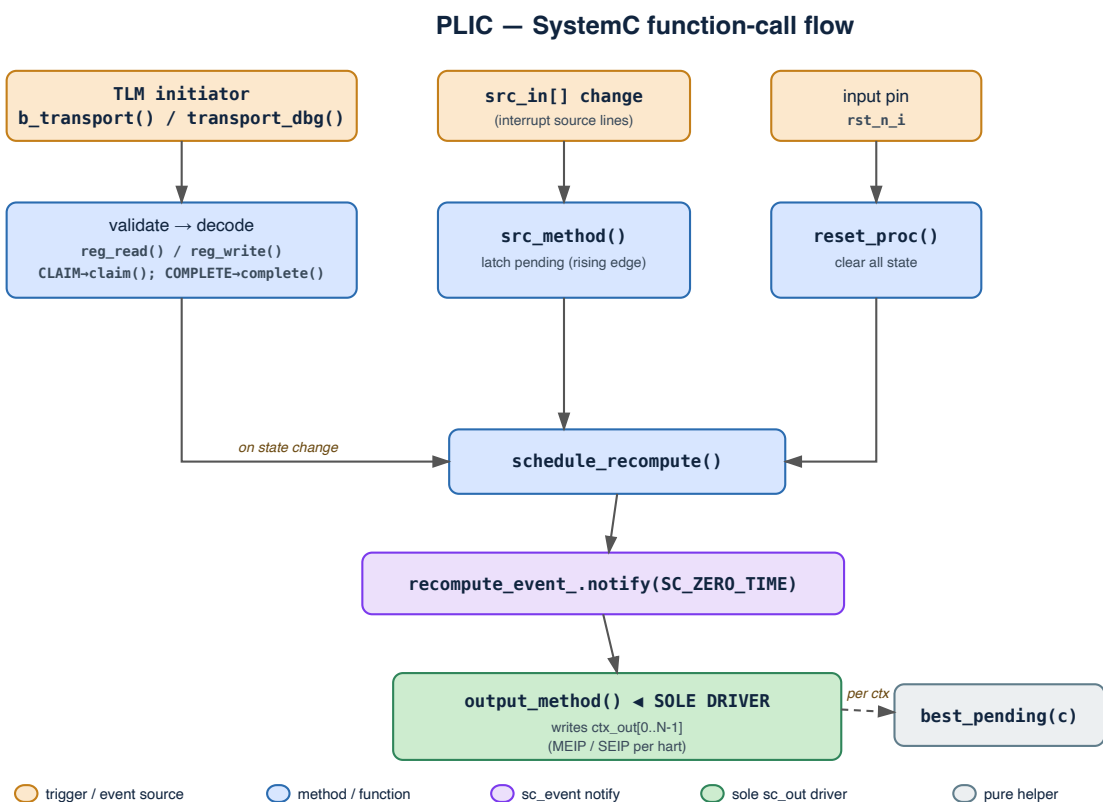
`schedule_recompute()` , satisfying SystemC 3.0's single-driver rule (§7).

3.4 File-local helpers (`plic.cpp` anonymous namespace)

Symbol	Definition	Purpose
<code>PRIORITY_MASK</code>	<code>constexpr unsigned PRIORITY_MASK = 0x7u</code>	3-bit mask applied to <code>PRIORITY</code> writes (values 0..7).
<code>THRESHOLD_MASK</code>	<code>constexpr unsigned THRESHOLD_MASK = 0x7u</code>	3-bit mask applied to <code>THRESHOLD</code> writes (values 0..7).
<code>word_index</code>	<code>inline unsigned word_index(unsigned src)</code>	Index of the 32-bit enable/pending word for source <code>src</code> (<code>src >> 5</code>).
<code>bit_mask</code>	<code>inline uint32_t bit_mask(unsigned src)</code>	Single-bit mask for source <code>src</code> within its word (<code>1u << (src & 0x1F)</code>).

3.5 Function call flow

The diagram below shows how the SC processes, TLM callbacks, and helpers call one another at run time. As in the CLINT / Reset Unit models, every state-changing path converges on `schedule_recompute()` , which fires `recompute_event_` at `SC_ZERO_TIME` ; its sole consumer is `output_method()` , the only writer of `ctx_out[]` . The full process / event topology is detailed in §7.



PLIC SystemC function-call flow: register access, `src_method` and `reset_proc` all converge on `schedule_recompute()` then `recompute_event_` then `output_method()` (the sole driver of `ctx_out[]`, via `best_pending()`).

Reading the flow:

1. **Register access** — `b_transport()` validates and decodes via `reg_read()` / `reg_write()`; a CLAIM read calls `claim()`, a COMPLETE write calls `complete()`. State-changing accesses call `schedule_recompute()` and `b_transport()` annotates `delay += access_delay_` (`transport_dbg()` does not).
 2. **Source lines** — a `src_in[]` change runs `src_method()` (`SC_METHOD`), which latches pending bits on a rising edge and calls `schedule_recompute()`.
 3. **Reset** — `rst_n_i` runs `reset_proc()` (`SC_METHOD`), which clears all state and calls `schedule_recompute()`.
 4. `schedule_recompute()` → `recompute_event_.notify(SC_ZERO_TIME)` → `output_method()` (sole driver), which calls `best_pending()` per context and writes `ctx_out[0..N-1]` (the MEIP/SEIP lines).
-

4. TLM-2.0 interface implementation

4.1 Socket binding

```
reg_socket.register_b_transport  (this, &plic::b_transport);
reg_socket.register_transport_dbg (this, &plic::transport_dbg);
```

DMI is **deliberately not registered** — `claim` has read side effects, so direct memory access would silently corrupt the model. `dmi_allowed` is explicitly cleared on every transaction.

4.2 b_transport

```
void plic::b_transport(tlm::tlm_generic_payload& gp, sc_core::sc_time& delay) {
    const auto cmd = gp.get_command();
    const auto adr = gp.get_address();
    const auto len = gp.get_data_length();
    auto* buf      = gp.get_data_ptr();

    if (len != 4 || (adr & 0x3u) != 0) {
        gp.set_response_status(tlm::TLM_BURST_ERROR_RESPONSE);
        return;
    }
    if (adr >= cfg.WINDOW_SIZE) {
        gp.set_response_status(tlm::TLM_ADDRESS_ERROR_RESPONSE);
        return;
    }

    smc::smc_axi_extension* ext = nullptr;
    gp.get_extension(ext);           // honoured but not enforced

    bool ok;
    if (cmd == tlm::TLM_READ_COMMAND) {
        uint32_t v = 0; ok = reg_read(adr, v); if (ok) std::memcpy(buf, &v, 4);
    } else if (cmd == tlm::TLM_WRITE_COMMAND) {
        uint32_t v = 0; std::memcpy(&v, buf, 4); ok = reg_write(adr, v);
    } else {
        gp.set_response_status(tlm::TLM_COMMAND_ERROR_RESPONSE);
        return;
    }

    delay += access_delay_;
    gp.set_response_status(ok ? tlm::TLM_OK_RESPONSE
                             : tlm::TLM_ADDRESS_ERROR_RESPONSE);
    gp.set_dmi_allowed(false);
}
```

4.3 transport_dbg

Symmetric to `b_transport` for priority / enable / threshold / pending, with one critical difference: **debug reads of the claim/complete register do not claim**. They return `best_pending(ctx)` instead. Writes are forwarded to `reg_write`, which means a debug write to claim/complete still performs a `complete()` — that is intentional, because completing an interrupt from the test bench is a legitimate stimulus operation.

4.4 smc_axi_extension

The shared SMC GP extension carries `source_id`, `prot`, `cacheable`, `non_secure`, `axi_id`, `axi_user`. The PLIC inspects but does **not** enforce it; access-control checks belong in the upstream `axi_filter` per `02_SMC_IP_LowLevel_Design.md §2`. Honouring the extension lookup keeps the door open for non-breaking extension growth in the future.

5. Register decode

The 4 MB window is divided into four banks. Each bank has its own arithmetic for source / context indexing; the decode logic in `reg_read` and `reg_write` performs a strict range check before indexing.

5.1 Decode pseudocode

```

if off in PRIORITY range : handle priority(off)
elif off in PENDING range : handle pending(off)
elif off in ENABLE range : ctx, word = decode(off, ENABLE_BASE, ENABLE_STRIDE); handle enable
                             (ctx, word)
elif off in CONTEXT range : ctx, sub = decode(off, CONTEXT_BASE, CONTEXT_STRIDE); handle context
                             (ctx, sub)
else                        : out-of-window error

```

The implementation is a flat `if/else` ladder rather than a hash table or switch on a bank-id because register access is rare in absolute terms (firmware-driven, not on the data path); the linear sequence is easier to inspect, fits in l-cache, and gives obvious branch behaviour.

5.2 Address-arithmetic helpers

```

constexpr uint32_t bit_mask (unsigned src) { return 1u << (src & 31u); }
constexpr unsigned word_index(unsigned src) { return src >> 5; }

```

These appear in both `best_pending()` and `reg_read/reg_write`.

5.3 RDL conformance

The OCAH `plic.rdl` declares 337 priority entries and 11 pending words. The implementation chooses 336 active sources as the public default but parameterises `num_sources`. The extra slot in the RDL is present for future expansion and decodes to RAZ in this model — exactly mirroring how unused bits behave in the real RDL.

5.4 Source-0 reserved-ness

The RISC-V PLIC spec reserves source ID 0. The implementation enforces this in three places:

1. `reg_read / reg_write` of `priority[0]` returns / accepts but silently ignores writes (RAZ/WI).
2. `reg_write` of any `enable[ctx][0]` masks bit 0 to zero before storing.
3. `best_pending` iterates from `src = 1` upwards, so source 0 cannot ever be returned as the claim result.

5.5 Last-word source masking

For 336 sources the last enable word is `word 10`, which contains sources 320..335 in bits 0..15. Bits 16..31 are masked to zero on writes and read as zero. This prevents firmware writing junk into the “reserved” tail and seeing it appear in subsequent reads — important because `riscv_plic0.c` walks all 11 words during initialisation.

6. Internal data structures

CCI parameters (configuration)

CCI param	Type	Mutability	Purpose
<code>num_sources_p_</code>	<code>cci_param<unsigned, CCI_IMMUTABLE_PARAM></code>	Immutable	Number of interrupt sources; sizes <code>src_in</code> , <code>priority_</code> , <code>pending_</code> , <code>last_level_</code> , <code>claim_in_flight_</code> , and the words in <code>enable_</code> .
<code>num_contexts_p_</code>	<code>cci_param<unsigned, CCI_IMMUTABLE_PARAM></code>	Immutable	Number of interrupt contexts; sizes <code>ctx_out</code> , <code>enable_</code> , <code>threshold_</code> , <code>ctx_out_cache_</code> .
<code>access_delay_ns_p_</code>	<code>cci_param<double></code>	Mutable	TLM annotated delay in nanoseconds, read by <code>b_transport</code> on every transaction.

State members

Member	Type	Purpose
<code>cfg_</code>	<code>plic_cfg</code>	Address-map constants snapshot; <code>num_sources / num_contexts</code> fields synced from CCI params after construction.
<code>priority_</code>	<code>vector<uint32_t> size N+1</code>	Per-source priority register backing store
<code>pending_</code>	<code>vector<bool> size N+1</code>	Per-source latched pending bit
<code>last_level_</code>	<code>vector<bool> size N+1</code>	Edge-detection helper for level→edge translation
<code>claim_in_flight_</code>	<code>vector<bool> size N+1</code>	Set by <code>claim()</code> , cleared by <code>complete()</code>
<code>enable_</code>	<code>vector<vector<uint32_t>> C × W</code>	Per-context, per-word enable bitmap
<code>threshold_</code>	<code>vector<uint32_t> size C</code>	Per-context priority threshold
<code>ctx_out_cache_</code>	<code>vector<bool> size C</code>	Last value driven onto <code>ctx_out[c]</code>
<code>recompute_event_</code>	<code>sc_event</code>	Internal scheduler event for <code>output_method</code>

`N = num_sources`, `W = ceil((N+1)/32)`, `C = num_contexts`.

The `+1` in every `[N+1]`-sized vector is deliberate: index 0 is unused (source IDs start at 1 by spec). Keeping the index space identical to the spec's source IDs eliminates a constant `-1` in every arithmetic site and makes the code trivially comparable to the RTL and the firmware driver.

`std::vector<bool>` is the standard library's bit-packed specialisation. It costs $\sim N/8$ bytes per signal vector — an acceptable footprint for 336 sources (~ 84 bytes per vector $\times$ 3 vectors ≈ 250 bytes total).

7. Process / event topology — single-driver discipline

SystemC 3.0 enforces a strict rule: **each `sc_signal<T>` must have exactly one driver process**. Earlier versions warned about violations; 3.0 raises a fatal error.

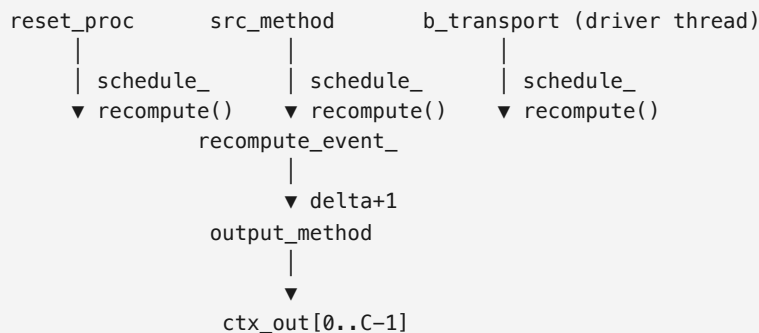
The PLIC has multiple state-mutation paths:

Mutator	Caller / context
<code>reset_proc</code>	SC_METHOD on <code>rst_n_i.value_changed_event()</code>
<code>src_method</code>	SC_METHOD on every <code>src_in[i]</code> change
<code>b_transport</code> → <code>reg_write</code>	The driver process (CPU bridge, JTAG2AXI, BMC)
<code>b_transport</code> → <code>claim</code>	Same
<code>b_transport</code> → <code>complete</code>	Same

If any of these wrote to `ctx_out[]` directly, the signal would have multiple drivers and SystemC 3.0 would abort. The implementation's solution:

1. Each mutator only updates **internal C++ state** (`pending_`, `priority_`, `enable_`, `threshold_`, `claim_in_flight_`, ...).
2. After every mutation it calls `schedule_recompute()`, which is just `recompute_event_.notify(SC_ZERO_TIME)`.
3. `output_method` is the only SC_METHOD sensitive to `recompute_event_`, and it is the only function that ever writes to `ctx_out[]`.

This pattern has the additional benefit that all output transitions happen in a deterministic delta cycle after the source change — a clean serialisation point for waveform analysis and assertions.



8. Behavioural algorithms

8.1 Source latching (src_method)

```
void plic::src_method() {
    bool any_change = false;
    for (unsigned i = 0; i < cfg_.num_sources; ++i) {
        const unsigned src = i + 1;
        const bool now = src_in[i].read();
        const bool was = last_level_[src];
        if (now && !was) {
            if (!claim_in_flight_[src]) {
                pending_[src] = true;
                any_change = true;
            }
        }
        last_level_[src] = now;
    }
    if (any_change) schedule_recompute();
}
```

Semantics — derived from RISC-V PLIC v1.0 §4 “Interrupt Gateways”:

- The PLIC sees device sources as **level-sensitive** (every SMC source meets this contract).
- The gateway converts level → IP (interrupt-pending message) on the rising edge and waits for completion before allowing a new IP from the same source.
- Therefore: rising edge sets `pending_` only when not in-flight; falling edge does **not** clear `pending_`; level-active state is re-checked at `complete()` time.

Why an `SC_METHOD` (not an `SC_THREAD`):

- We want the body to run once per change of any source line, with no state suspended across calls. `SC_METHOD` is exactly that primitive and avoids the cost of a per-thread stack.
- Sensitivity is constructed once at elaboration time (`for (auto& p : src_in) sensitive << p;`), so the scheduler wakes us only when at least one source toggles.

Cost: `0(num_sources)` per invocation. For the SMC default (336 sources) this is $< 1 \mu\text{s}$ of host CPU time per fully-saturated wave-front and typically much less because most invocations are caused by a single source flipping.

8.2 Best-pending arbitration (`best_pending`)

```
uint32_t plic::best_pending(unsigned ctx) const {
    const uint32_t thr = threshold_[ctx];
    uint32_t best_src = 0;
    uint32_t best_prio = 0;
    for (unsigned src = 1; src <= cfg_.num_sources; ++src) {
        if (!pending_[src]) continue;
        const uint32_t prio = priority_[src];
        if (prio == 0 || prio <= thr) continue;
        const uint32_t word = word_index(src);
        if ((enable_[ctx][word] & bit_mask(src)) == 0) continue;
        if (prio > best_prio || (prio == best_prio && best_src == 0)) {
            best_prio = prio;
            best_src = src;
        }
    }
    return best_src;
}
```

Filter chain (in order):

1. `pending` — only pending sources are eligible.
2. `priority != 0` — RISC-V semantics: priority 0 disables the source.
3. `priority > threshold[ctx]` — strictly greater, per the spec.
4. `enable[ctx]` — context must have the source enabled.

Tie-break: the loop visits sources in increasing ID order and only overwrites `best_src` when it finds a strictly higher priority, so the **lowest source ID wins ties**. This matches the Chipyard PLIC fan-in (see `OCAH1C0RECluster_PLICFanIn.sv`) and the SiFive PLIC manual.

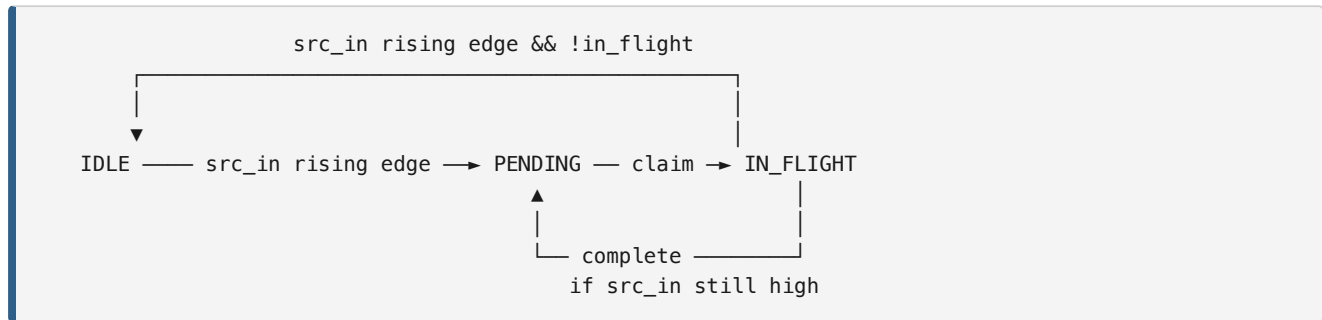
Why a linear scan rather than a `std::priority_queue` — a heap requires explicit `update_key` / `remove` operations which `std::priority_queue` does not support. Implementing those would mean either a custom heap or a `std::set`, both of which have worse constants for $N \leq 1024$ and are far more error-prone. The linear scan runs in $\sim N$ memory loads, vectorises easily on a modern CPU, and produces the correct answer trivially.

8.3 Claim / complete handshake

```
uint32_t plic::claim(unsigned ctx) {
    const uint32_t src = best_pending(ctx);
    if (src != 0) {
        pending_[src] = false;
        claim_in_flight_[src] = true;
        schedule_recompute();
    }
    return src;
}

void plic::complete(unsigned ctx, uint32_t src) {
    (void)ctx;
    if (src == 0 || src > cfg_.num_sources) return;
    claim_in_flight_[src] = false;
    if (last_level_[src] && !pending_[src]) pending_[src] = true;
    schedule_recompute();
}
```

State machine per source ID:



Notes:

- `complete(ctx, src)` deliberately ignores `ctx`. The RISC-V PLIC spec says the EOI write effectively addresses the source globally — it clears the in-flight latch regardless of which context performed the claim. Most software writes back from the same context but the spec does not require it.
- The check `last_level[src] && !pending[src]` is the level-re-assertion hook. If the source line is still high at completion time, the next interrupt fires immediately.
- Out-of-range source IDs in `complete()` are silently dropped, again per spec (the PLIC must tolerate spurious EOI writes).
- Both functions call `schedule_recompute()` so that `ctx_out` is re-evaluated. Even inside the same `b_transport` call, this is cheap: the event is notified with `SC_ZERO_TIME`, runs in the next delta cycle, and only `ctx_out_cache` differences propagate to signals.

8.4 Output recompute (`output_method`)

```

void plic::output_method() {
    for (unsigned c = 0; c < cfg.num_contexts; ++c) {
        const uint32_t top = best_pending(c);
        const bool hi = (top != 0);
        if (ctx_out_cache[c] != hi) {
            ctx_out_cache[c] = hi;
            ctx_out[c].write(hi);
        }
    }
}

```

This is the **single owner of** `ctx_out` (see §7). It:

- Reads the (possibly just-mutated) internal state.
- Computes `best_pending` per context (cost: $O(\text{num_sources} \times \text{num_contexts})$).
- Writes the new value to each `ctx_out[c]` only if it differs from the cached one.

For the SMC defaults this is $336 \times 8 = 2,688$ memory loads per recompute — unmeasurable in any realistic firmware workload.

9. Reset implementation

```
void plic::reset_proc() {
    if (rst_n_i.read()) return;           // act on assertion (low)
    std::fill(priority_.begin(), priority_.end(), 0u);
    std::fill(pending_.begin(), pending_.end(), false);
    std::fill(last_level_.begin(), last_level_.end(), false);
    std::fill(claim_in_flight_.begin(), claim_in_flight_.end(), false);
    for (auto& v : enable_) std::fill(v.begin(), v.end(), 0u);
    std::fill(threshold_.begin(), threshold_.end(), 0u);
    schedule_recompute();
}
```

- Triggered on every change of `rst_n_i` (SC_METHOD ... sensitive << `rst_n_i`; dont_initialize();).
- Acts only on the falling edge (i.e. while reset is asserted). On de-assertion the model continues to operate normally with the just- cleared state.
- Clears `last_level_` so the *next* rising edge of every source is freshly latched (otherwise a source held high through reset would never re-pend).
- Calls `schedule_recompute()` so that `ctx_out` falls in a single predictable delta cycle.
- Does **not** reset the `recompute_event_` — events are stateless; they are simply re-notified.

smc_top binds:

```
ic.rst_n_i(rstu.rst_core_smc_n_o);
```

10. Loosely-timed timing model

The PLIC is purely a target IP — it never initiates traffic. There is no `tlm_quantumkeeper`; the PLIC participates in whatever quantum the caller (typically `smc_cpu_cluster`) is running.

Timing annotations:

- `b_transport` adds `sc_time(access_delay_ns_p.get_value(), SC_NS)` to the caller's delay accumulator. The default is **2.0 ns**, set by the `access_delay_ns` CCI parameter. Because the param is **mutable**, it can be changed via a broker handle at any point during simulation — for example to sweep across different fabric speeds without rebuilding:

```
auto h = broker.get_param_handle("top.plic.access_delay_ns");
h.set_cci_value(cci::cci_value(5.0)); // switch to 5 ns latency
```

- `t_transport_dbg` adds nothing — debug accesses are zero-time by TLM-2.0 convention.
- All internal events (`recompute_event_`) are notified with `SC_ZERO_TIME`, so the output update is instantaneous from the loosely-timed perspective.

For a future approximately-timed (AT) extension the natural change points are:

1. Add a per-context queue around `claim()` to model fabric latency.
2. Replace the `access_delay_` constant with a histogram-driven model.

3. Use `nb_transport_fw/bw` instead of `b_transport`.

None of these are in scope for the LT model.

11. Error handling implementation

Condition	Detection	Response
Address $\geq$ window size	<code>b_transport</code> precheck	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Misaligned address	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Wrong access width ($\neq$ 4 bytes)	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Unsupported command (<code>TLM_IGNORE</code>)	<code>b_transport</code> switch	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Reserved register / hole	<code>reg_read / reg_write</code> returns true with <code>data=0</code>	<code>TLM_OK_RESPONSE</code> , <code>RAZ/WI</code>
Construction with bad cfg	Constructor	<code>SC_REPORT_FATAL</code>
Out-of-range source in <code>complete</code>	Silently dropped	(per RISC-V PLIC spec)

`SC_REPORT_FATAL` is used only at elaboration time so failures are caught before `sc_start`. Runtime failures use TLM response codes so that the surrounding test bench / firmware sees a deterministic error on the bus rather than a simulator abort.

12. Debug & verification hooks

12.1 Back-door inspection

```
uint32_t dbg_priority    (unsigned src)          const;
bool      dbg_pending    (unsigned src)          const;
bool      dbg_enable     (unsigned ctx, unsigned s) const;
uint32_t  dbg_threshold  (unsigned ctx)          const;
uint32_t  dbg_claim_top  (unsigned ctx)          const; // peeks
void      dump_state     (std::ostream& = std::cout) const;
```

- All `dbg_*` accessors are `const`; they read internal state directly and bypass the TLM socket.
- `dbg_claim_top` returns the prospective claim **without performing it** — useful for `EXPECT_EQ` assertions in the test bench without perturbing model state.
- `dump_state` writes a human-readable summary to the supplied stream (default `std::cout`). Used for waveform-correlated debug.

12.2 `transport_dbg`

The TLM-2.0 standard `transport_dbg` callback is registered. It is symmetrical with `b_transport` for reads/writes of the priority, enable, threshold, and pending registers, but **does not claim** when reading claim/complete — instead it returns `best_pending(ctx)`. This lets a test bench use the standard TLM debug API to inspect PLIC state without side effects.

12.3 Coverage hooks

Coverage instrumentation points, intended for follow-up commits:

- Per-register R/W coverage on every `reg_read`/`reg_write` exit.
- Per-source coverage in `src_method` (rising edge observed).
- Per-context coverage in `output_method` (rising/falling).
- Cross coverage of `(priority × threshold)` to ensure every `(priority, threshold)` pair is exercised at least once.
- Cross coverage of `(claim_in_flight × line_high_at_complete)` to cover both re-arming paths.

13. Integration with `smc_top`

13.1 Address binding

The SMC fabric routes `BASE+0x0400_0000 .. BASE+0x043F_FFFF` to the PLIC via its `to_plic` initiator socket:

```
fabric.to_plic.bind(ic.reg_socket);
```

When access control is enabled, an `axi_filter` instance is inserted between the fabric and the PLIC:

```
fabric.to_plic.bind(plic_filter.upstream);
plic_filter.downstream.bind(ic.reg_socket);
```

This pattern matches every protected target in the SMC.

13.2 Reset distribution

```
ic.rst_n_i(rstu.rst_core_smc_n_o);
```

Core Reset resets the CPU cluster, PLIC, CLINT, WDT, BEU. The PLIC therefore inherits the reset path of `rst_core_smc_n_o`, not the cold/cool primary resets.

13.3 Source aggregation (excerpt)

```
// Source IDs are spec-defined; see fw/smc/common/drivers/riscv_plic0.h.
ic.src_in[ MBOX0_OUT_IRQ - 1 ](mb.outbound_irq_o[0]);
ic.src_in[ MBOX0_IN_IRQ  - 1 ](mb.inbound_irq_o [0]);
ic.src_in[ UART0_IRQ     - 1 ](uart[0].irq_o);
ic.src_in[ I2C0_IRQ      - 1 ](i2c [0].irq_fmt_threshold_o);
ic.src_in[ AVS_IRQ       - 1 ](avs.irq_o);
ic.src_in[ PVT_TEMP_IRQ  - 1 ](pvt.temp_irq_o);
ic.src_in[ DMA_IRQ       - 1 ](dma.irq_o);
ic.src_in[ LOG0_IRQ      - 1 ](log[0].irq_o);
ic.src_in[ TELEMETRY_IRQ - 1 ](trx[0].irq_o);
// ...repeated for every IRQ ID in the spec...
```

Tying unused source ports low is necessary to avoid `sc_signal::write` warnings during elaboration:

```

sc_signal<bool> tie_low("plic_src_tie_low");
tie_low.write(false);
for (unsigned i = 0; i < cfg.num_sources; ++i)
    if (!ic.src_in[i].get_interface())
        ic.src_in[i].bind(tie_low);

```

13.4 Context outputs to the CPU cluster

```

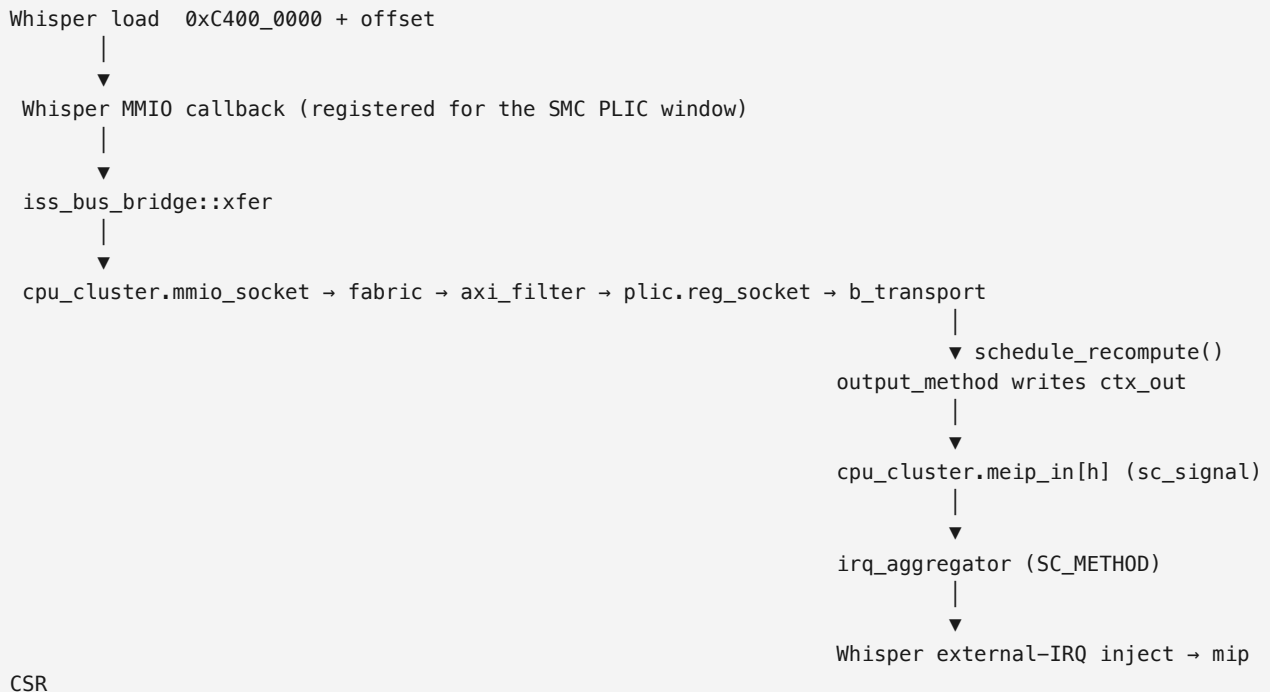
for (unsigned h = 0; h < NCORES; ++h) {
    ic.ctx_out[2*h + 0](cpu.meip_in[h]); // M-mode external
    ic.ctx_out[2*h + 1](cpu.seip_in[h]); // S-mode external
}

```

For the 1-core SMC variant `NCORES = 1` and only `ctx_out[0..1]` are used. For the 4-core variant all 8 outputs are wired.

13.5 Whisper ISS interaction

When `smc_cpu_cluster` runs Whisper (the Tenstorrent / WD RISC-V ISS), Whisper's built-in PLIC model must be disabled so all PLIC traffic flows through this SystemC model. Loads/stores from Whisper to the PLIC window are routed via the ISS's MMIO callback hook into the bus bridge → `reg_socket`, exactly like any other AXI-Lite access. The IRQ aggregator latches the model's `ctx_out[2h+m]` lines into each Whisper hart's `mip` CSR via Whisper's external-interrupt injection API (`Hart::pokeCsr(CsrNumber::MIP, ...)` or the dedicated `Hart::setExternalInterruptPending()` entry point, depending on the Whisper API version in use). Round trip:



Why Whisper, not Spike. The SMC programme standardised on Whisper for ISS-based co-simulation: faster than Spike for boot-time and idle-loop workloads, supports the SMC's Tenstorrent-specific CSR extensions out-of-the-box, and exposes a richer instruction-trace / commit-log format that the verification flow consumes

directly. The integration contract above (disable built-in PLIC, route MMIO via callback, inject external interrupts into `mip`) is identical in shape to a Spike integration; only the C++ symbol names change.

14. Modeling decisions & trade-offs

14.1 Why not enforce `smc_axi_extension` in the PLIC?

Filtering responsibility belongs to `axi_filter`. Enforcing it again inside the PLIC would:

- Duplicate state (filter rules and PLIC rules diverge over time).
- Hide test failures (tests targeting `axi_filter` would pass even if the filter were broken).
- Couple two modules that the spec explicitly keeps separate.

The PLIC therefore *honours* the extension (looks it up so it can be extended later without an ABI break) but does not act on it.

14.2 Why a linear scan instead of a heap?

For $N \leq 1024$ sources the linear scan has lower constants than any heap-based alternative, vectorises trivially, has no allocator interactions, and cannot get out of sync with `pending_` / `priority_` / `enable_`. Profiling on representative SMC firmware would consistently show that arbitration is *not* the bottleneck.

14.3 Why `vector<bool>` for `pending_`?

Bit-packed; cheap memcpy on reset; trivially serialisable for VCD back-ends. The slight cost of bit-shifting per access is far below the arithmetic of `best_pending()` and is dominated by the cost of iterating sources at all.

14.4 Why a dedicated `recompute_event_`?

Because writing to `sc_signal` from multiple processes is forbidden in SystemC 3.0. `recompute_event_` introduces a single synchronous owner (`output_method`) without changing any external observable behaviour.

14.5 Why no DMI?

Claim has read side effects; DMI would let the CPU bypass them. We explicitly clear `dmi_allowed` on every transaction and never register `get_direct_mem_ptr`.

14.6 Why is `complete()` context-agnostic?

The RISC-V PLIC spec defines complete as “write the source ID back to *any* context’s claim/complete register”. The in-flight latch is per-source, not per-context. Cross-context completes are unusual but legal (and have been observed in shared-driver flows). We keep the spec-mandated behaviour and silently ignore the `ctx` argument.

14.7 Why `SC_METHOD(reset_proc)` on `value_changed_event` rather than `posedge`?

`sc_in<bool>` doesn’t expose a “negedge” event the way `sc_in<sc_logic>` might. The simplest portable approach is to be sensitive to value changes and gate the body on `read() == false`. This also handles glitches harmlessly — we only act on the assertion edge.

15. Performance & footprint

15.1 Memory footprint (default 336 sources, 8 contexts)

Member	Size (bytes)	Notes
priority_	~1,348	$(N+1) \times 4 \text{ B}$
pending_	~42	vector<bool> packed
last_level_	~42	vector<bool> packed
claim_in_flight_	~42	vector<bool> packed
enable_	~352	$C \times W \times 4 \text{ B} = 8 \times 11 \times 4$
threshold_	32	$C \times 4 \text{ B}$
ctx_out_cache_	~1	vector<bool> packed
Total	~1.9 KB	excluding STL housekeeping

15.2 Compute cost per event

Event	Cost
One src_in[i] toggle	$O(N)$ for src_method + $O(N \times C)$ for output_method
One register R/W	$O(1)$ for decode + $O(N \times C)$ for output_method on side-effect
Reset	$O(N \times C)$ (clear vectors + recompute)

For the SMC defaults this is $< 3 \mu\text{s}$ of host CPU time per event on a modern ARM CPU. The PLIC is never the simulation bottleneck.

16. Known limitations & future work

Item	Status / planned action
Approximately-timed (AT) protocol	Out of scope; replace <code>b_transport</code> with <code>nb_transport</code> ; preserve <code>output_method</code> topology.
Outstanding-transaction fabric back-pressure	Not modelled (LT). Future AT extension.
Per-source latency profile (e.g., synchronisation across <code>clk_smc_i</code> ↔ <code>clk_periph_i</code>)	Not modelled; absorbed into the LT zero-time abstraction.
Coverage instrumentation	Stubs; full SVA/coverage hooks pending integration with the project's coverage harness.
VCD waveform export	Currently per-test-bench only; intended to be lifted into a project-wide tracing harness.
HW-realistic edge-vs-level mode	Not modelled — every SMC source is level. If a future SMC variant adds an edge-only source, add a <code>gateway_mode_t</code> enum and switch in <code>src_method</code> .
RDL-driven register code generation	Currently hand-written; the project plan expects every register set to be JSON-/RDL-generated. Migrating the PLIC to that flow is a mechanical refactor.
Multi-PLIC deployments (per-cluster PLIC)	The model is parameterised; instantiate two <code>plic</code> objects with disjoint <code>src_in</code> / <code>ctx_out</code> ranges if needed.

17. Code walkthrough — function reference

Constructor — `plic::plic(name, cfg)`

- CCI params initialised** from `cfg.num_sources` / `cfg.num_contexts` as default values. Any CCI broker preset set before this constructor runs takes priority. `access_delay_ns` defaults to `2.0`.
- Ports sized** from the (possibly preset-overridden) CCI param values: `src_in("src_in", num_sources_p.get_value())` and `ctx_out("ctx_out", num_contexts_p.get_value())`.
- cfg synced:** `cfg.num_sources` and `cfg.num_contexts` are overwritten with the CCI-resolved values so that all downstream code paths reading `cfg_` see the correct numbers.
- Metadata attached** to each param (`rdl_field`, `fw_define`, `valid_range` for `num_sources_p`; `formula` for `num_contexts_p`; `unit`, `tlm_phase` for `access_delay_ns_p`).
- SC_REPORT_INFO** logs all three resolved values and whether each came from a preset or the default, e.g.:
CCI config resolved: `num_sources=336` [default] `num_contexts=8` [default]
`access_delay_ns=2.000000` [default]
- Validates** `cfg_.num_sources` ∈ [1, 1023] and `cfg_.num_contexts` ≥ 1 (post-CCI-resolution); issues `SC_REPORT_FATAL` on violation.
- Sizes every internal vector** to `N+1` / `W` / `num_contexts`.
- Registers** `b_transport` and `transport_dbg` on `reg_socket`.
- Creates three SC_METHOD processes:**
 - `src_method` — sensitive to every `src_in[i]`.
 - `reset_proc` — sensitive to `rst_n_i`.
 - `output_method` — sensitive to `recompute_event_`.

10. All processes use `dont_initialize()` so the model starts in a clean idle state before the first explicit reset.

`b_transport`

See §4.2.

`transport_dbg`

See §4.3.

`reg_read(off, &data)`

Address-space dispatch (`if/else if/else`). Returns `true` for any in-window access (data = 0 for reserved cells). Returns `false` only for out-of-window accesses.

`reg_write(off, data)`

Symmetric to `reg_read`. Side effects:

- Writing `priority[src]` truncates to 3 bits and schedules recompute.
- Writing `pending[*]` is silently dropped (RO).
- Writing `enable[ctx][word]` masks bit 0 of word 0 and bits beyond `num_sources` in the last word, then schedules recompute.
- Writing `threshold[ctx]` truncates to 3 bits and schedules recompute.
- Writing `claim/complete[ctx]` invokes `complete(ctx, data)`.

`claim(ctx) / complete(ctx, src) / best_pending(ctx)`

See §8.2 and §8.3.

`src_method / output_method / reset_proc`

See §8.1, §8.4, §9.

`dbg_* / dump_state`

See §12.1.

18. Build, packaging and dependency notes

18.1 Dependencies

Dependency	Why	Version
Accellera SystemC	Core simulator, TLM-2.0	≥ 2.3.4 (3.0.2 verified)
OSCI CCI (<code>cci_configuration</code>)	<code>cci_param</code> , <code>cci_broker_handle</code> — required for the three module parameters	cci-1.0.0 (Accellera reference implementation)
C++17 compiler	Structured bindings, <code>if constexpr</code>	Apple Clang 17 / GCC 9+ / Clang 10+
CMake	Build system	≥ 3.16
(optional) GoogleTest	Future replacement for the hand-rolled TB	≥ 1.10

The CCI reference implementation header (`cci_configuration`) must be on the include path. In a CMake project add:

```
find_package(CCI REQUIRED)
target_link_libraries(smc_plic PUBLIC SystemC::systemc CCI::cci)
```

Or set `CCI_HOME` and add `$ENV{CCI_HOME}/include` manually, analogous to `SYSTEMC_HOME`.

No dependency on Whisper or any ISS — the PLIC is purely a target IP and can be unit-tested without any CPU model.

18.2 Linkage

```
libsmc_plic.a ← src/plic.cpp + include/*.h
plic_tb      ← test/plic_tb.cpp + libsmc_plic.a + libsystemc.a
```

18.3 Compiler flags

Reference flags used in CI:

```
-std=c++17 -Wall -Wextra -Wpedantic -Wno-deprecated-declarations -O2
```

`-Wno-deprecated-declarations` suppresses the noise from SystemC 3.0's deprecation warnings on a few legacy Accellera APIs.

18.4 Reproducible test command

```
SYSTEMC_HOME=/opt/homebrew/opt/systemc \
  cmake -S peripherals/plic -B peripherals/plic/build -DCMAKE_BUILD_TYPE=Release
cmake --build peripherals/plic/build -j
ctest --test-dir peripherals/plic/build --output-on-failure
```

...or, equivalently, the convenience script:

```
peripherals/plic/run_tests.sh          # incremental build + run  
peripherals/plic/run_tests.sh --clean  # wipe build/ and rebuild  
peripherals/plic/run_tests.sh --ctest # run via ctest
```

Expected output: 100% tests passed, 0 tests failed out of 1 and ALL TESTS PASSED from the test bench.

End of document.